

ANALYSIS AND DESIGN OF ALGORITHMS

Instructor: Dr. Mert Büyükdede



Single-source shortest-paths

Single-source shortest-paths problem is defined as follows.

For a given vertex called the source in a weighted connected graph, the problem is to find shortest paths to all its other vertices.

The single-source shortest-paths problem asks for a family of paths, each leading from the source to a different vertex in the graph, though some paths may, of course, have edges in common.

In a weighted graph, the **single source shortest path** problem aims to find, from a given **source vertex**, the shortest paths to all other vertices.

Given a graph $G=(V,E)$ and an edge-weight function $w:E\rightarrow\mathbb{R}$ one selects for each target vertex the path whose total weight is minimal.

This problem is a fundamental component in many applications, from transportation networks to communication protocols.



Applications and Importance

- **Route Finding & Navigation:** Employed in mapping and GPS services to determine the shortest or fastest route. For example, the OSPF (Open Shortest Path First) routing protocol uses Dijkstra's algorithm to build a shortest-path tree for each router.
- **Network Routing:** Distance-vector protocols like RIP (Routing Information Protocol) share and update cost vectors by applying a Bellman–Ford–style relaxation.
- **Optimization Problems:** Core to project planning (PERT charts), logistics, transportation networks, robotic path planning, and even machine-learning graph models. For instance, optimizing the order in which tasks are visited or goods are delivered often reduces to repeated single-source shortest-path computations.

We generally use two methods to solve the shortest path problem: Dijkstra and Bellman-Ford.

Dijkstra Problem

Dijkstra's algorithm, introduced by Edsger W. Dijkstra in 1959, efficiently computes the shortest path from a chosen source vertex to all other vertices in a non-negative, weighted graph. It is widely used in network routing, geographic information systems, and many optimization problems.

Beyond its theoretical appeal, Dijkstra's algorithm has been embedded in many real-world systems. In Internet routing protocols such as OSPF (Open Shortest Path First) and IS-IS, routers use variations of Dijkstra's method to compute efficient forwarding tables.

In mapping and navigation applications, the algorithm computes driving or walking directions by treating road segments as weighted edges.

Moreover, adaptations of Dijkstra's core idea appear in resource-allocation problems, scheduling tasks with precedence constraints, and in certain dynamic programming formulations.



Problem Statement

Given:

- A directed or undirected graph $G=(V,E)$
- A non-negative weight function $w:E \rightarrow \mathbb{R}_{\geq 0}$
- A source vertex $s \in V$

Find:

- For each vertex $v \in V$ the length $d[v]$ of the shortest path from s to v .
- Optionally, the predecessor $\pi[v]$ for reconstructing the path.

Distance estimate $d[v]$: current best known distance from s to v .

Predecessor $\pi[v]$: the vertex immediately preceding v on the current shortest path.

Priority queue Q : holds all vertices not yet finalized, keyed by $d[v]$. A binary heap or Fibonacci heap may be used.

Working Process

- Dijkstra's algorithm finds the shortest paths to a graph's vertices in order of their distance from a given source.

Initialization:

Assign each vertex v a tentative distance value: $d[v] \leftarrow \infty$.

Set the source's distance to zero: $d[s] \leftarrow 0$.

Mark all vertices as “unvisited.”

Place all vertices into a min-priority queue keyed by $d[v]$.

Select Next Vertex:

Extract (remove) from the queue the unvisited vertex u with the smallest tentative distance $d[u]$.

Mark u as “visited” (its shortest distance is now finalized).



Working Process

Edge Relaxation

For each neighbor v of u (each edge (u,v)):

 Compute alternative distance: $alt = d[u] + w(u,v)$.

 If $alt < d[v]$, then:

 Update $d[v] \leftarrow alt$.

 Record predecessor $\pi[v] \leftarrow u$.

 Decrease v 's key in the priority queue to the new $d[v]$.

Repeat

Continue extracting the next smallest-distance unvisited vertex and relaxing its edges until the queue is empty.

Termination

- Once every vertex has been visited (queue is empty), all $d[v]$ values are the lengths of the shortest paths from s .
- The predecessor map π lets you reconstruct each actual path by walking backwards from any target v to s .

Path Reconstruction (optional)

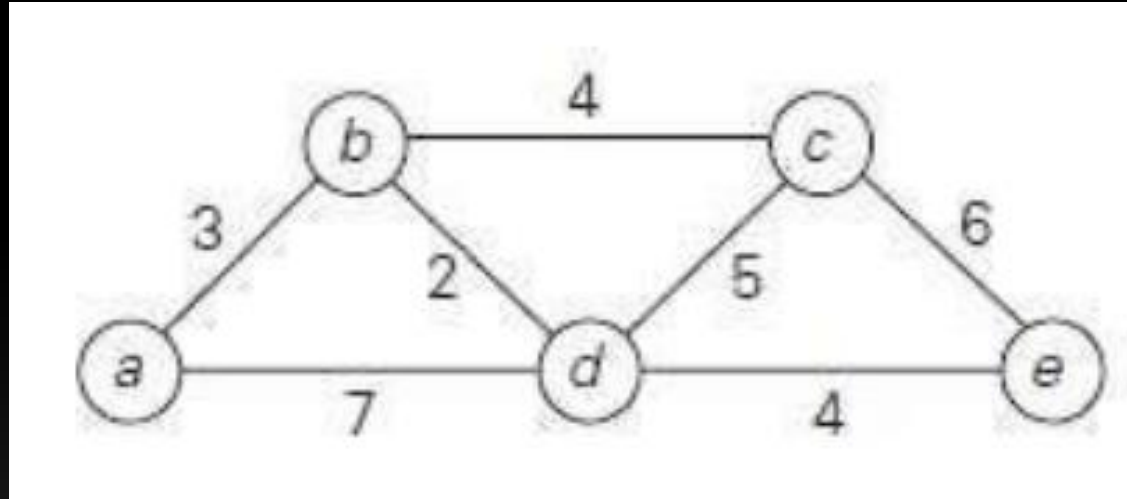
1. To retrieve the path from s to a given vertex t :
 1. Start at t , follow $\pi[t]$, ... until you reach s .
 2. Reverse this sequence to obtain the path in the correct order.

Complexity Notes

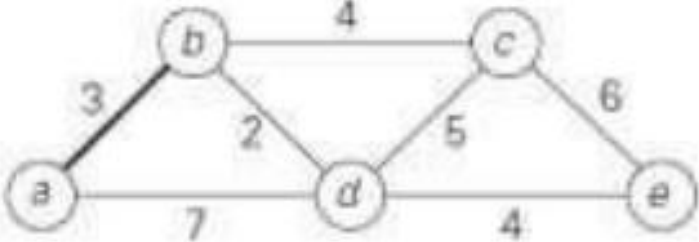
1. Using a binary heap: $O((|V|+|E|) \log |V|)$.
2. Using a Fibonacci heap: $O(|V| + \log |V| + |E|)$.

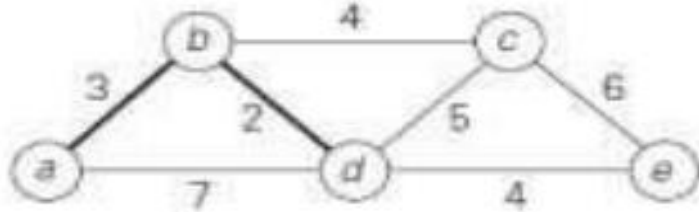
This sequence of initialization, greedy selection, relaxation, and repetition ensures that each vertex's final distance label is the true shortest-path length.

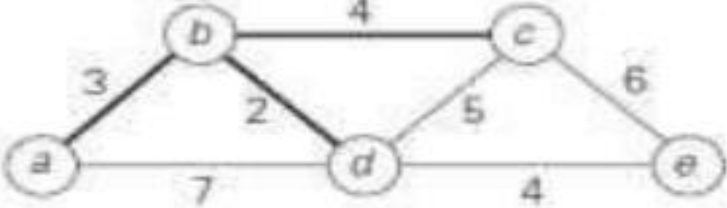
Illustration: An example of Dijkstra's algorithm is shown below.

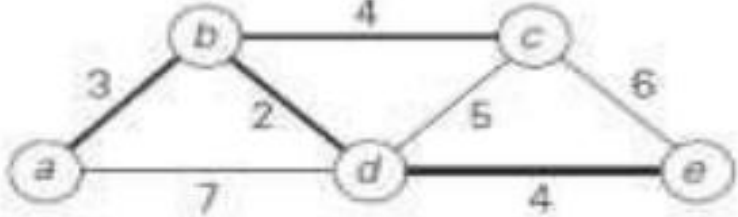


The shortest paths (identified by following nonnumeric labels backward from a destination vertex in the left column to the source) and their lengths (given by numeric labels of the tree vertices) are as follows:

Tree vertices	Remaining vertices	Illustration
$a(-, 0)$	$b(a, 3) \quad c(-, \infty) \quad d(a, 7) \quad e(-, \infty)$	

Tree vertices	Remaining vertices	Illustration
$b(a, 3)$	$c(b, 3 + 4) \quad d(b, 3 + 2) \quad e(-, \infty)$	

Tree vertices	Remaining vertices	Illustration
$d(b, 5)$	$c(b, 7) \quad e(d, 5 + 4)$	

Tree vertices	Remaining vertices	Illustration
$c(b, 7)$	$e(d, 9)$	

Let's write Algorithm of Dijkstra Problem with Pseudo Code

ALGORITHM *Dijkstra*(G, s)

//Dijkstra's algorithm for single-source shortest paths

//Input: A weighted connected graph $G = \langle V, E \rangle$ with nonnegative weights

// and its vertex s

//Output: The length d_v of a shortest path from s to v

// and its penultimate vertex p_v for every vertex v in V

Initialize(Q) //initialize priority queue to empty

for every vertex v in V

$d_v \leftarrow \infty$; $p_v \leftarrow \text{null}$

Insert(Q, v, d_v) //initialize vertex priority in the priority queue

$d_s \leftarrow 0$; *Decrease*(Q, s, d_s) //update priority of s with d_s

$V_T \leftarrow \emptyset$

for $i \leftarrow 0$ **to** $|V| - 1$ **do**

$u^* \leftarrow \text{DeleteMin}(Q)$ //delete the minimum priority element

$V_T \leftarrow V_T \cup \{u^*\}$

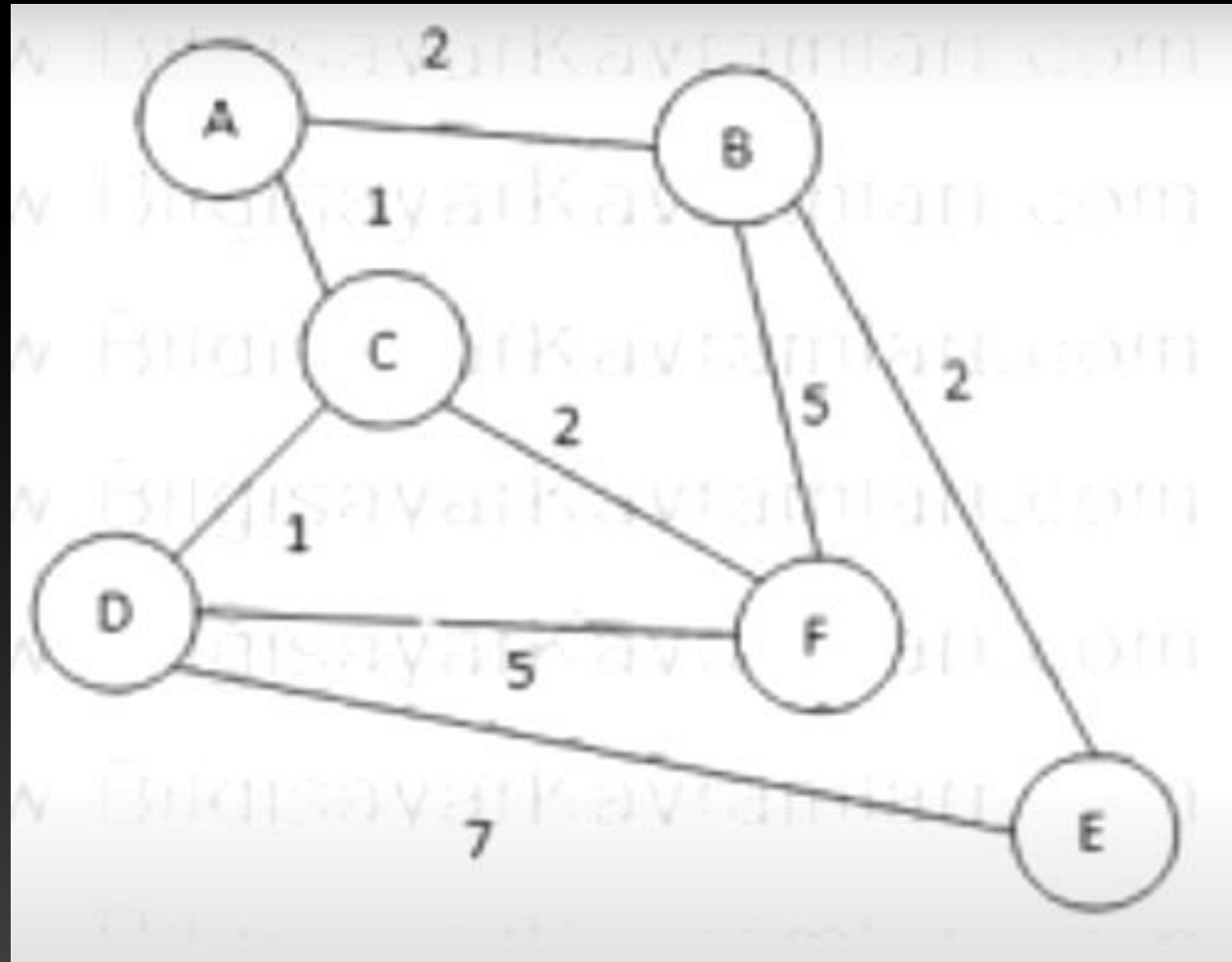
for every vertex u in $V - V_T$ that is adjacent to u^* **do**

if $d_{u^*} + w(u^*, u) < d_u$

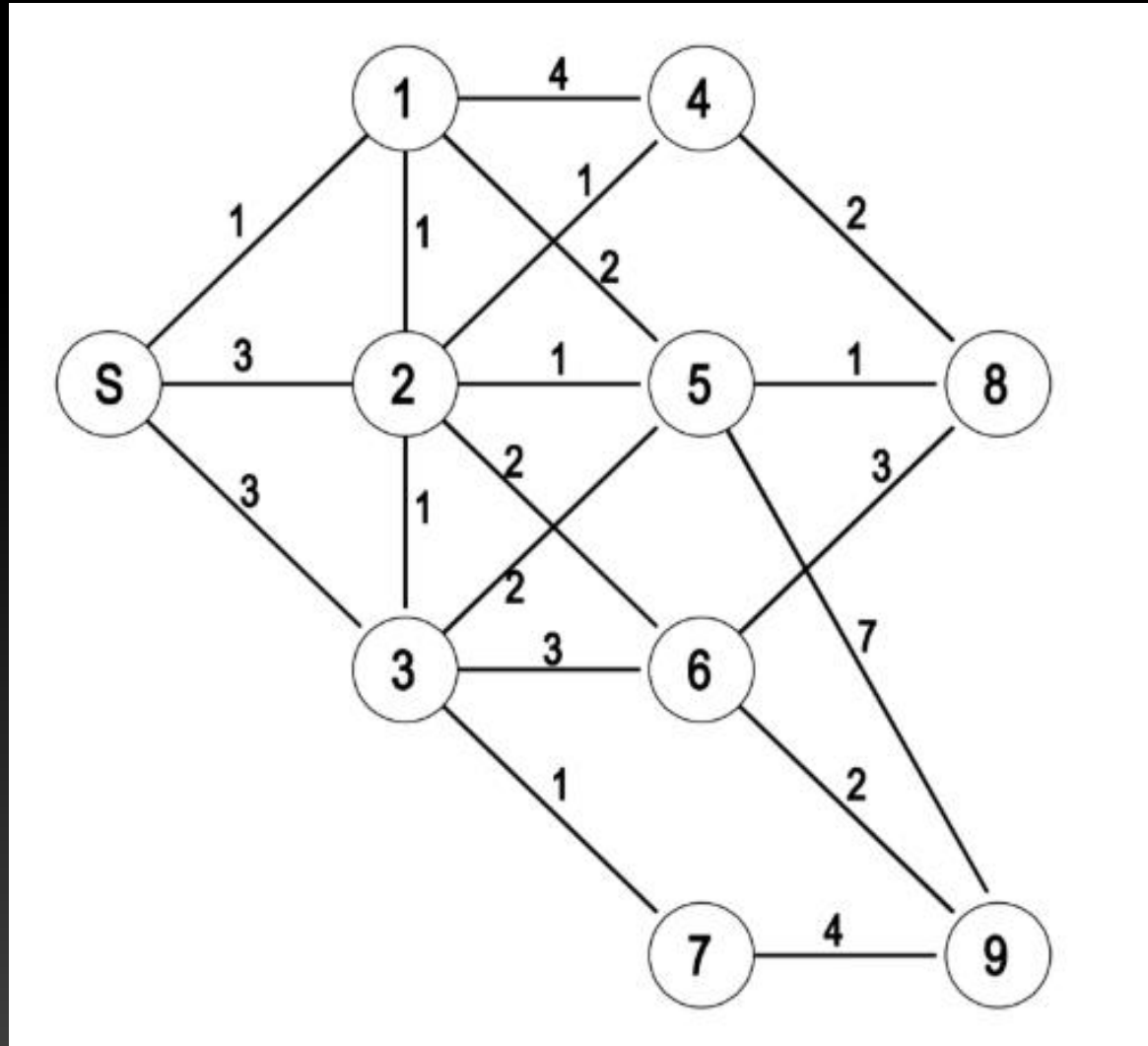
$d_u \leftarrow d_{u^*} + w(u^*, u)$; $p_u \leftarrow u^*$

Decrease(Q, u, d_u)

Example



Example



Activity Scenario: “Istanbul EMS Route Planner”

In the midst of peak afternoon traffic in Istanbul, the Emergency Medical Services (EMS) must dispatch an ambulance from the Central Hospital (H) to a critical incident in Sultanahmet (S) as quickly as possible. Eight key junctions—Central Hospital (H), Kadıköy (K), Beşiktaş (B), Üsküdar (U), Taksim (T), Sultanahmet (S), Eminönü (E), and Ortaköy (O)—experience varying traffic loads.

Based on recent traffic studies, the average travel times (in minutes) between these points have been estimated. EMS headquarters will model these locations as nodes in a weighted graph, with edges labeled by their travel times. The goal is to apply Dijkstra’s algorithm to determine the fastest route from H to S.

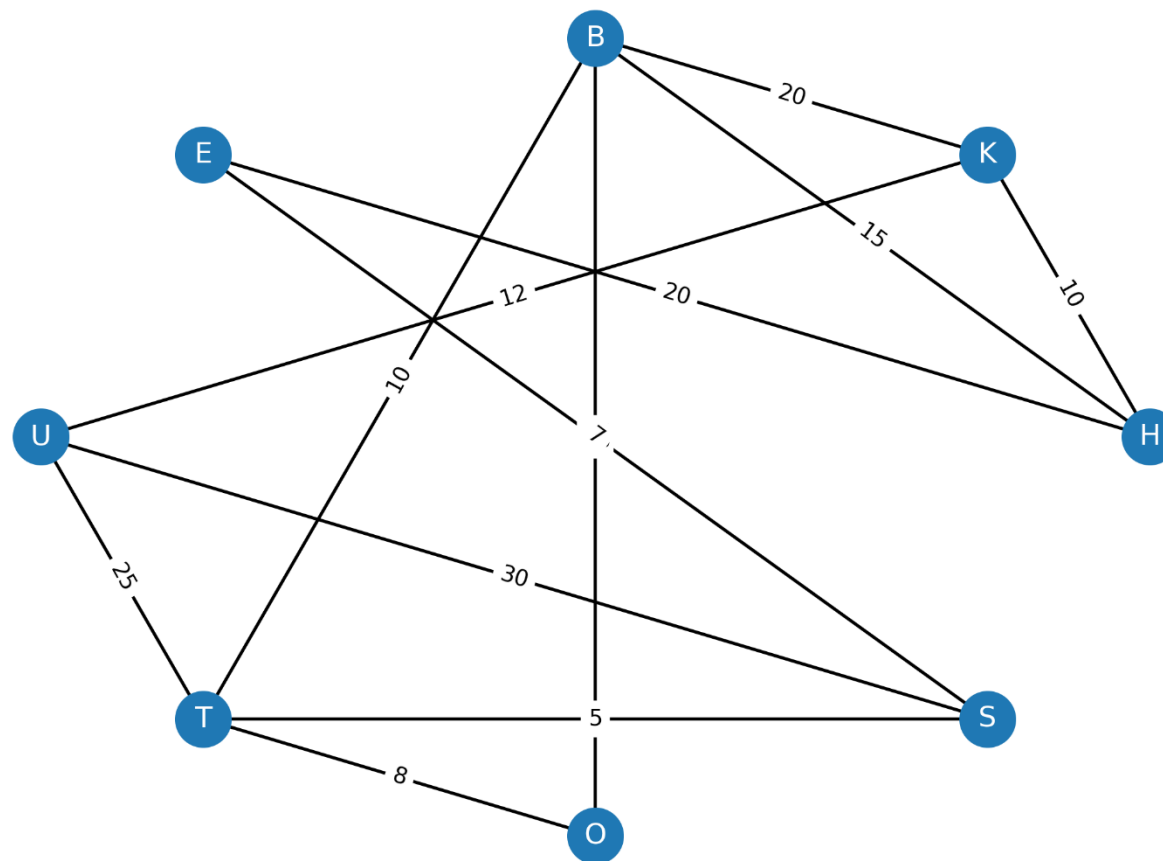


Phase I: Graph Drawing

Let's draw a graph according to the table given below.

From → To	H	K	B	U	T	S	E	O
H	—	10	15	—	—	—	20	—
K	10	—	20	12	—	—	—	—
B	15	20	—	—	10	—	—	18
U	—	12	—	—	25	30	—	—
T	—	—	10	25	—	5	—	8
S	—	—	—	30	5	—	7	—
E	20	—	—	—	—	7	—	—
O	—	—	18	—	8	—	—	—

Istanbul EMS Route Planner - Weighted Graph



Phase 2: Pseudocode Writing

Let's try to create Algorithm with Pseudo Code for this problem.

```
DIJKSTRA(Graph G, source H):  
    // 1. Initialization  
    for each vertex v in G.vertices:  
        dist[v]  $\leftarrow \infty$            // Tentative distance from H  
        pred[v]  $\leftarrow$  NIL           // Predecessor on the shortest path  
    dist[H]  $\leftarrow$  0                 // Distance to source is zero  
  
    // 2. Build min-priority queue Q keyed by dist[]  
    Q  $\leftarrow$  new MinPriorityQueue()  
    for each v in G.vertices:  
        Q.insert(v, dist[v])
```

Phase 2: Pseudocode Writing

Let's try to create Algorithm with Pseudo Code for this problem.

```
// 3. Main Loop
while Q is not empty:
    u ← Q.extractMin()           // Vertex with smallest dist[]
    for each Edge (u,v) in G.adj[u]:
        alt ← dist[u] + weight(u,v) // Candidate distance via u
        if alt < dist[v]:         // Relaxation step
            dist[v] ← alt
            pred[v] ← u
            Q.decreaseKey(v, alt)

// 4. Return results
return dist, pred
```

HOMEWORK Phase 3: Java Code Implementation

Let's try to write Java Code for this problem.



Bellman–Ford Algorithm

The Bellman–Ford algorithm is used to find the shortest paths from a single source to all nodes. Unlike Dijkstra, it allows for negative weights on edges; however, it can also detect a negative cycle, as the concept of a “shortest path” becomes meaningless.

This algorithm’s key characteristics are:

1.Support for Negative Edge Weights

Unlike Dijkstra’s algorithm, Bellman–Ford allows edge weights to be negative. This makes it suitable for scenarios in which costs may be refunded or gains modeled as negative values.

2.Negative-Cycle Detection

If a graph contains a cycle whose total weight is negative, the concept of a “shortest path” becomes ill-defined because traversing the cycle repeatedly can continuously reduce the path length. Bellman–Ford performs an additional pass to detect any such negative cycles and reports their presence.

3.General Applicability

The algorithm works on both directed and undirected graphs and can be readily integrated into applications where the number of vertices and edges is moderate.



Bellman–Ford Algorithm

Application areas include route planning, network flow analysis, automated risk assessment in financial models, and more complex optimization problems requiring dynamic programming. In particular, its reliability in handling negative weights makes Bellman–Ford widely useful.

In the next section, we will delve into the mathematical foundations of the algorithm and explain, step by step, how the relaxation process operates.



The Process of Bellman–Ford Algorithm

1. Set up distances

1. Give the source node s , a distance of 0.
2. Give every other node “infinity” (unknown) as its distance.

2. Relax edges repeatedly

1. Repeat the following step $(|V| - 1)$ times:
 1. For each edge $(u \rightarrow v)$, see if going $s \rightarrow \dots \rightarrow u \rightarrow v$ is shorter than the current known distance to v .
 2. If it is, update v 's distance and remember that its best predecessor is u .

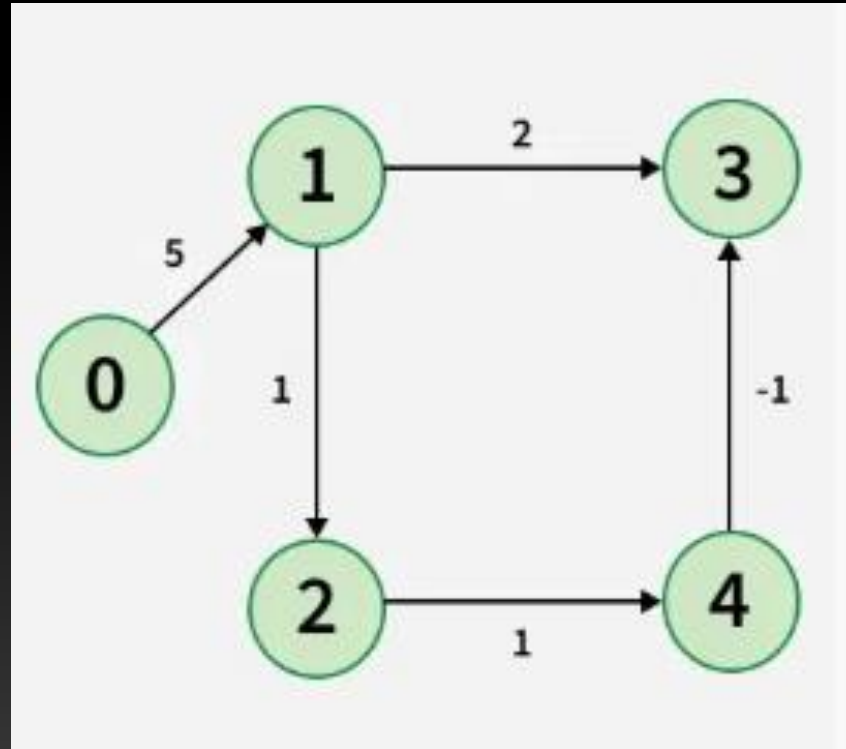
3. Check for negative cycles

1. Do one more pass over all edges.
2. If any distance can still be reduced, there's a reachable negative-weight cycle.
3. If not, all distances are final and correct.

Comparison: Dijkstra vs. Bellman–Ford

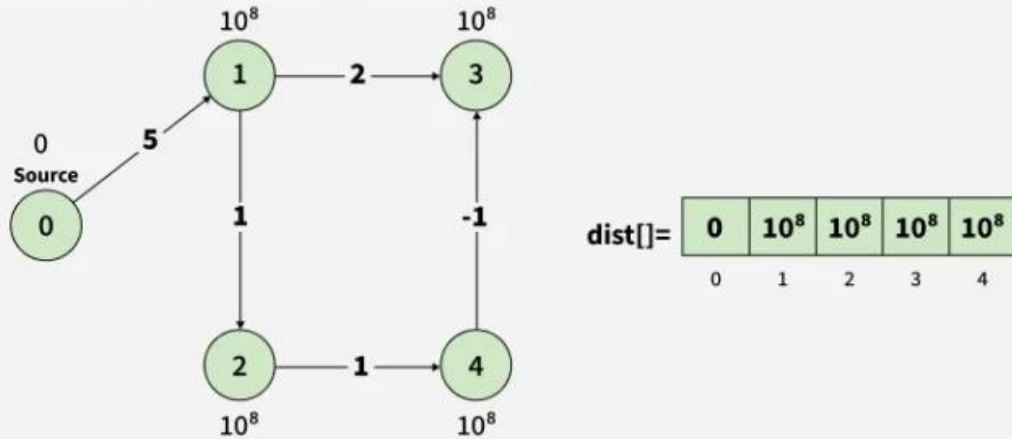
Aspect	Dijkstra	Bellman–Ford
Edge Weights	Nonnegative only	Arbitrary (including negative)
Negative-Cycle Handling	✗ (cannot detect)	✓ (detects and reports cycles)
Time Complexity	$O(E \log V)$ (binary heap)	$O(VE)$
Approach	Greedy (extract-min + relax)	Systematic edge-relaxation loop
Common Uses	GPS/navigation, OSPF routing, robotics	RIP routing, financial arbitrage

Example of Bellman–Ford Algorithm



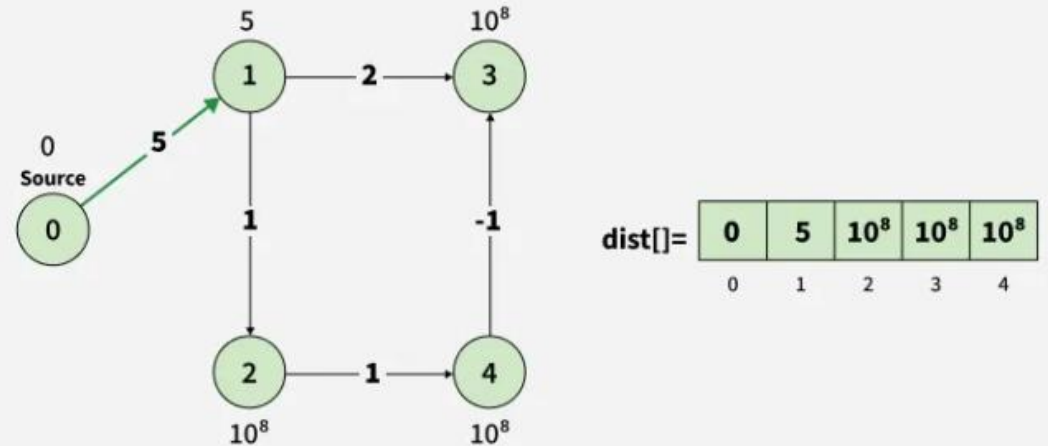
01
Step

Maintain a `dist[]` array to store the minimum distance from the source node to every other vertex. Initialize `dist[source] = 0`, since the distance from the source to itself is zero, and set all other `dist[]` values to 10^8 .



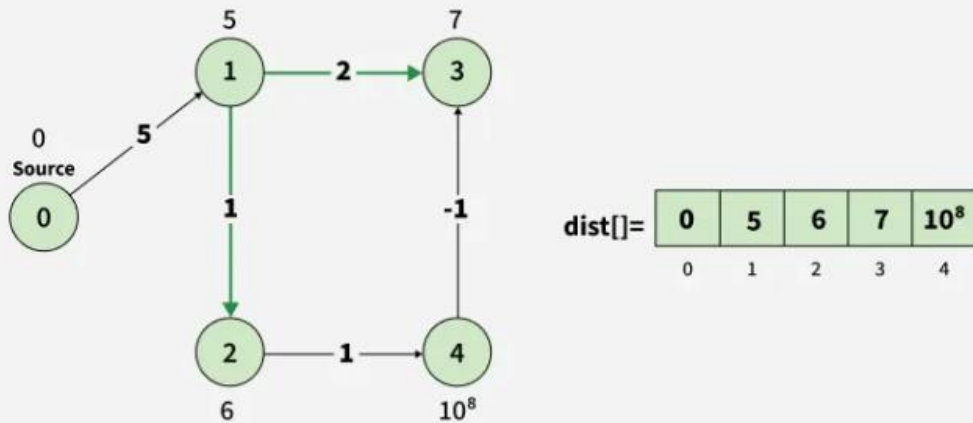
02
Step

In the first relaxation step, iterate through all edges and apply the relaxation formula: `dist[v] > dist[u] + weight(u, v)`. If the condition holds, update `dist[v]` accordingly.



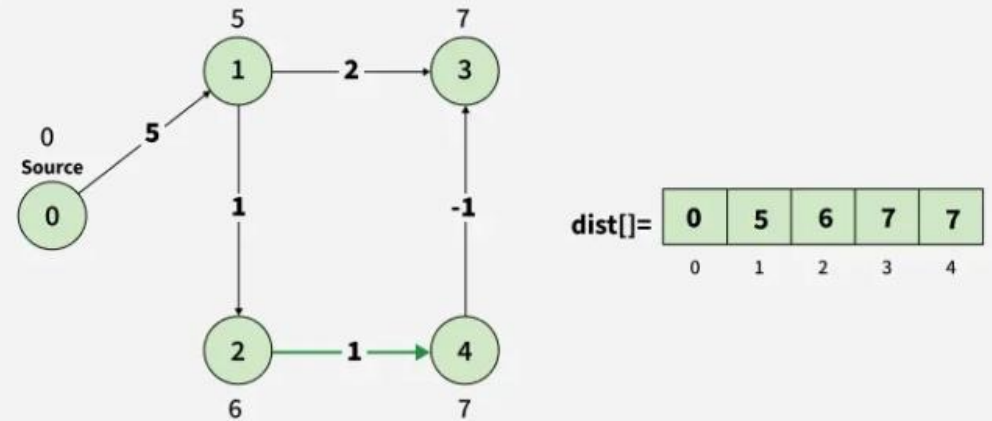
03
Step

In the Second relaxation step, iterate through all edges and apply the relaxation formula: `dist[v] > dist[u] + weight(u, v)`. If the condition holds, update `dist[v]` accordingly.



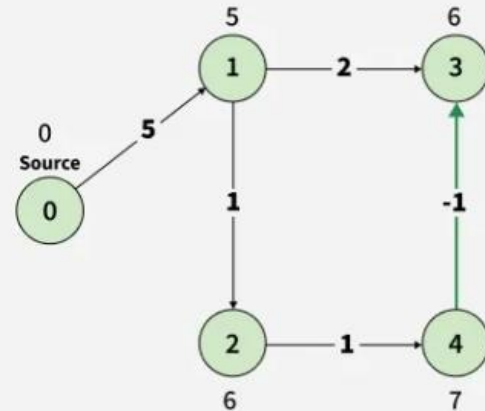
04
Step

In the Third relaxation step, iterate through all edges and apply the relaxation formula: `dist[v] > dist[u] + weight(u, v)`. If the condition holds, update `dist[v]` accordingly.



05
Step

In the Fourth relaxation step, iterate through all edges and apply the relaxation formula: $\text{dist}[v] > \text{dist}[u] + \text{weight}(u, v)$. If the condition holds, update $\text{dist}[v]$ accordingly

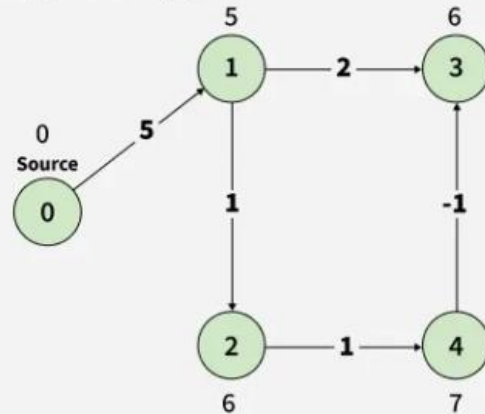


$\text{dist}[] =$

0	5	6	6	7
0	1	2	3	4

06
Step

In the fifth relaxation, In the Fourth relaxation step, iterate through all edges and apply the relaxation formula: $\text{dist}[v] > \text{dist}[u] + \text{weight}(u, v)$. If the condition holds, update $\text{dist}[v]$ accordingly.



$\text{dist}[] =$

0	5	6	6	7
0	1	2	3	4

In a graph without negative cycles, the shortest path can have at most $V-1$ edges. So any change in the V -th iteration indicates a negative cycle.

Let's write Algorithm of Bellman-Ford with Pseudo Code

```
BELLMAN-FORD( $G, w, s$ )  
  // 1. Initialization  
  for each vertex  $v$  in  $G.V$  do  
     $d[v] \leftarrow \infty$   
     $\pi[v] \leftarrow \text{NIL}$   
   $d[s] \leftarrow 0$   
  
  // 2. Relax edges repeatedly  
  for  $i \leftarrow 1$  to  $|G.V| - 1$  do  
    for each edge  $(u \rightarrow v)$  in  $G.E$  do  
      if  $d[u] + w(u, v) < d[v]$  then  
         $d[v] \leftarrow d[u] + w(u, v)$   
         $\pi[v] \leftarrow u$   
  
  // 3. Check for negative-weight cycles  
  for each edge  $(u \rightarrow v)$  in  $G.E$  do  
    if  $d[u] + w(u, v) < d[v]$  then  
      report "Negative-weight cycle detected"  
  
  // 4. Return results  
  return  $(d, \pi)$ 
```

Activitiy: Scenario

A regional logistics company operates out of a **Central Depot (A)** and must plan delivery routes to several facilities across the area. To encourage traffic through less-traveled roads, the company offers **rebates** (negative costs) on certain routes.

Vertices (Locations):

- A: Central Depot
- B: Warehouse North
- C: Warehouse East
- D: Retail Outlet City Centre
- E: Retail Outlet Suburb
- F: Rural Station
- G: Maintenance Hub
- H: Emergency Supply Point

Directed Edges with Costs

From	To	Cost
A	B	3
A	C	6
B	C	2
B	D	5
B	H	3
C	E	4
C	A	-4
D	E	1
D	G	3
D	H	4
E	F	-3
E	G	2
F	B	-2
F	D	7
G	A	1
G	F	-1
H	C	-2

Student Tasks:

Step 1: Graph Construction

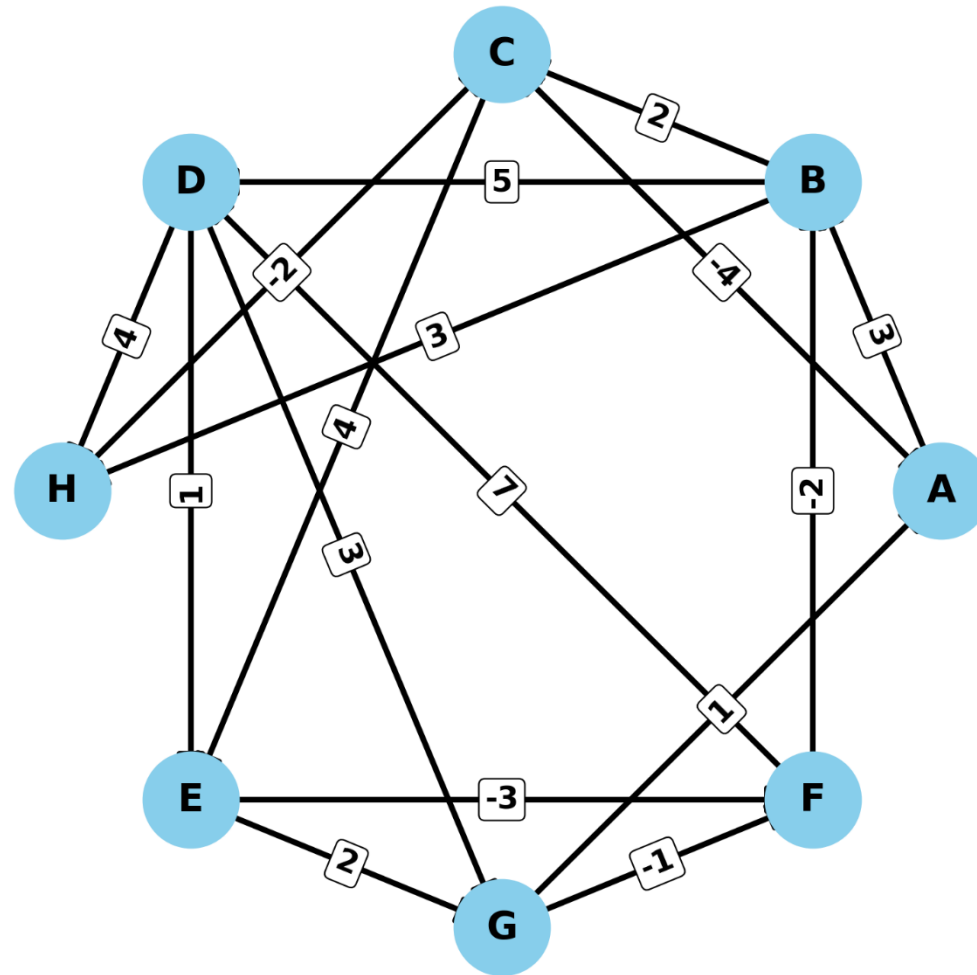
- Using the expanded dataset, draw the directed, weighted graph on paper or a whiteboard.
 - Label each node with its location code (A–H).
 - For every route, draw an arrow from **From** to **To** and write its **Cost** alongside.
 - Clearly mark negative-cost edges to show rebate routes.

Step 2: Pseudocode Design

- Write clear pseudocode for the Bellman–Ford algorithm

Step 1: Graph Construction

Logistics Network Graph (Circular Layout)



Step 2: Pseudocode Design

```
// Pseudocode for Bellman-Ford on the Logistics Network Scenario
```

```
procedure BellmanFordLogistics():
```

```
    // 0. Define the network
```

```
    vertices  $\leftarrow$  {A, B, C, D, E, F, G, H}
```

```
    edges  $\leftarrow$  [
```

```
        (A  $\rightarrow$  B, 3), (A  $\rightarrow$  C, 6),
```

```
        (B  $\rightarrow$  C, 2), (B  $\rightarrow$  D, 5), (B  $\rightarrow$  H, 3),
```

```
        (C  $\rightarrow$  E, 4), (C  $\rightarrow$  A, -4),
```

```
        (D  $\rightarrow$  E, 1), (D  $\rightarrow$  G, 3), (D  $\rightarrow$  H, 4),
```

```
        (E  $\rightarrow$  F, -3), (E  $\rightarrow$  G, 2),
```

```
        (F  $\rightarrow$  B, -2), (F  $\rightarrow$  D, 7),
```

```
        (G  $\rightarrow$  A, 1), (G  $\rightarrow$  F, -1),
```

```
        (H  $\rightarrow$  C, -2)
```

```
    ]
```

Step 2: Pseudocode Design

```
// 1. Initialization
for each v in vertices do
    distance[v]  $\leftarrow$   $+\infty$ 
    predecessor[v]  $\leftarrow$  NIL
distance[A]  $\leftarrow$  0           // Source is the Central Depot A

// 2. Relax all edges |V|-1 times
for i from 1 to |vertices| - 1 do
    for each (u  $\rightarrow$  v, w) in edges do
        if distance[u] + w < distance[v] then
            distance[v]  $\leftarrow$  distance[u] + w
            predecessor[v]  $\leftarrow$  u

// 3. Negative-cycle detection
for each (u  $\rightarrow$  v, w) in edges do
    if distance[u] + w < distance[v] then
        report "Negative-weight cycle detected (rebate loop)"
        return FAILURE

// 4. Output results
for each v in vertices do
    print "Cost from A to", v, "=", distance[v],
        "via", path reconstructed from predecessor[]

return SUCCESS
```

•**HOMEWORK Step 3: Java Implementation**

- Create a Java class (e.g., BellmanFordRebate)

